Humans Were the Bug

From Vibe Coding to Agent-Native Engineering

humanlint Project • github.com/jeppsontaylor/humanlint

Abstract

AI coding changes the bottleneck in software engineering. The scarce act is no longer typing plausible code; agents can produce plausible code in many mainstream languages. The scarce act is trustworthy merge: the repository must reject wrong generated code quickly, localize the defect, prove the repair, preserve security posture, and teach the next agent what happened.

This paper defines agent-native engineering as repositories designed as verification interfaces for generated code. It turns the earlier manifesto into an enforceable standard: conformance levels, stable rule identifiers, generated-zone policy, audit output, CI modes, repair queues, versioned artifacts, and a threat model for false plausibility, hostile context, tool overreach, secrets, dependency poisoning, and false-green tests.

The stack recommendation remains sharp but narrower. Repositories do not need humanlint merely because they use AI. Repositories claiming humanlint conformance need a control plane strong enough to prove it. Under the rubric in this paper, the default winner for durable product engineering is Rust core, TypeScript/React/Vite product surface, PostgreSQL durable truth, generated contracts, and bounded Python for AI/data service work. Exceptions are explicit, versioned, and auditable rather than hidden in taste or folklore.

Index Terms

AI-assisted software engineering, agentic coding, repository architecture, Rust, TypeScript, PostgreSQL, software audit, verification, code generation.

I. New Bottleneck: Trustworthy Merge

The title is intentionally blunt, but the claim is precise. Humans were not the bug because humans wrote software. Humans were the bug because most repositories were organized around human memory, human patience, human taste, and slow human review. That organization worked only while code arrived slowly enough for teams to compensate with folklore and senior-engineer recall. Agentic coding removes that throttle.

GitHub's 2025 Octoverse report describes an ecosystem where AI is mainstream in development and TypeScript has become the most-used language on GitHub [1]. DORA's 2025 research frames AI as an amplifier of existing organizational capability rather than an automatic productivity guarantee [2]. Security evidence is less forgiving: Veracode reports that generated code can be syntactically plausible while still containing known flaws [3], and GitGuardian reports accelerating secret exposure in public GitHub activity, including AI-assisted commits [4], [5].

The correct response is not to slow agents down until the old review process survives. The correct response is to redesign the repository as the control surface that constrains, proves, and repairs generated code. Agent-native engineering is not "humans approve whatever the model says." It is human-directed engineering in which the repo makes the invalid path hard to express and the valid path cheap to prove.

This paper makes four claims. First, programming languages are bottleneck-compression systems, not merely syntax preferences. Second, once agents can learn syntax on demand, language choice shifts toward rejection speed, security, proof locality, and auditability. Third, those qualities need a repository control plane before stack preference matters. Fourth, after the control plane is present, the strongest general-purpose default stack is Rust core, TypeScript/React/Vite, PostgreSQL, generated contracts, and bounded Python.

II. Definitions and Threat Model

The paper uses operational definitions rather than rhetorical labels. *Vibe coding* is any development mode where plausible code is accepted without explicit ownership, proof routing, generated-boundary checks, security gates, and repair evidence. *Agent-native engineering* is the inverse: the repository is structured so an agent can discover the owner, edit the smallest legal surface, run the mapped proof lane, and leave evidence. A *proof lane* is a deterministic command or CI job tied to a path or change class. A *generated zone* is an output path that is repaired by changing its source contract and regenerating. A *repair receipt* is the durable record of what failed, why, what was changed, and which proof accepted the repair. *Centerline drift* is the distance between the repository's actual structure and the standard it claims to follow.

The threat model starts with false plausibility. Agents can produce code that looks idiomatic, compiles in isolation, and still violates architecture, security, data ownership, or runtime assumptions. DORA's amplifier finding matters here: AI improves strong systems and worsens weak ones [2]. The repo must therefore treat generated code as a hypothesis until proof lanes accept it.

The standard does not claim every repository using AI must adopt humanlint. It claims a narrower and testable thing: a repository claiming humanlint conformance must expose enough machine-readable structure for the claim to be audited. Conformance is a public interface, not a vibe.

III. Languages as Bottleneck Compression

Programming language history is often told as a progression toward abstraction. That is incomplete. A language is a compression package: it decides which burdens move from human memory into syntax, type systems, runtimes, libraries, tooling, and conventions. Machine code compressed nothing. Assembly compressed opcodes into names. Fortran compressed scientific arithmetic. COBOL compressed business records. C compressed portable systems programming into a thin abstraction over hardware. Java and .NET compressed memory management and enterprise portability into managed runtimes. Python compressed exploration and glue. TypeScript compressed large-team JavaScript maintenance by adding a static feedback loop to the browser ecosystem, a direction now strengthened by Microsoft's native TypeScript 7 effort [6].

The agent-era lesson is that syntax is no longer the scarce resource. A model can imitate syntax. It cannot reliably infer unstated ownership, hidden exception policy, or tribal boundaries. The language and repository must make invalid paths structurally harder.

IV. Technical Promise Versus Standard Gravity

New languages emerge when an existing compression package stops matching the dominant pain. They fracture around runtime ceilings, ecosystem gaps, tooling weakness, migration cost, hiring constraints, interop friction, and fashion that lacks enforcement. A language can be technically excellent and still fail as a default company architecture if it lacks the operational surround: boring deployment, security scanning, observability, package maturity, database migration norms, and enough public examples for agents and humans to learn from.

Julia is the cleanest example of legitimate promise without universal default status. Its multiple dispatch, JIT compilation, and scientific-computing focus attack the two-language problem in numerical work: prototype in a high-level language, then rewrite hot paths in C, C++, or Fortran [9]. That is a real problem, and Julia remains a serious answer for many scientific domains. The adoption lesson is not that Julia is bad. The lesson is that solving an inner-loop expression problem does not automatically solve company-scale product engineering, cloud operations, security review, hiring, packaging, and cross-language contract discipline.

The shelf is not a graveyard of bad ideas. It is evidence that technical elegance and default-standard status are different things. Agent-native engineering rewards stacks where examples, tools, contracts, tests, and deployment conventions are abundant enough that agents can be constrained rather than merely prompted.

TABLE I
THREAT MODEL FOR AGENT-GENERATED CODE.

Threat	Failure mode	humanlint control
False plausibility	Code looks correct but violates hidden ownership, security, or data rules.	Owner map, test map, small files, generated contracts, typed errors.
Hostile context	README snippets, issues, web pages, docs, or comments instruct the agent to bypass policy.	Root and local agent instructions, permission policy, source-trust hierarchy.
Tool overreach	Agent edits generated files, secrets, unrelated owners, or broad directories.	Generated-zone manifest, path ownership, diff scope, CI mode.
Secret exposure	Prompted code commits tokens, logs secrets, or weakens scanners.	Secret scan, safe error payloads, no secrets in UI or docs, repair receipts.
Dependency poisoning	Agent adds unreviewed packages or abandons lockfile discipline.	SCA, SBOM/provenance, dependency rationale, update policy.
False-green tests	Tests pass because the wrong lane ran, assertions are weak, or mocks hide boundary drift.	Path-routed proof lanes, contract tests, Playwright critical paths, release matrix.

TABLE II
LANGUAGE EVOLUTION AS BOTTLENECK COMPRESSION.

Era or family	Human bottleneck compressed	Agent-era interpretation
Machine code and assembly	Remembering opcodes, addresses, registers, and jumps.	Exactness without reviewability is not enough; agent patches need semantic structure.
Fortran and COBOL	Expressing domain work without writing every machine operation.	Domain vocabulary matters because agents need stable concepts to route changes.
C and Unix systems programming	Portable control over hardware, memory, and operating-system interfaces.	Power without structural memory safety increases audit burden; CISA/NSA guidance pushes teams toward memory-safe defaults [7].
Java and .NET	Managed memory, deployment portability, enterprise libraries, and tooling.	Mature ecosystems help, but framework surface can hide ownership boundaries.
Python, R, Julia	Exploration, data analysis, notebooks, and scientific productivity.	Excellent for model/data loops; dangerous when unbounded into product truth. Python 3.14 improves runtime options, but does not change the boundary problem [8].
JavaScript and TypeScript	Browser programming, product velocity, and typed frontend maintenance.	Essential product surface; should not become durable backend truth.
Rust and modern safe systems languages	Memory safety, ownership, concurrency discipline, and explicit error handling.	Strong core for generated code because invalid states can be rejected before runtime.

V. VIBE-CODING FAULT TAXONOMY

Vibe coding is not a mood. It is a fault class: shipping code whose correctness depends on plausibility, confidence, or reviewer intuition instead of machine-checkable proof. The risk-priority number below is **RPN = impact $\times$ likelihood $\times$ detection difficulty**. It is a humanlint policy weight, not a universal incident statistic. Future releases should calibrate it against repair success, escaped defects, security regressions, and review time.

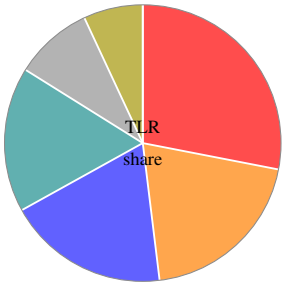
Public evidence supports treating vibe coding as a high-risk production pattern rather than a harmless workflow style. Veracode reports that AI-generated samples introduced risky security flaws in 45% of tests [3]. GitGuardian reports accelerating hardcoded-secret exposure in public GitHub activity and AI-service leak growth [4], [5]. Stack Overflow's 2025 survey reports that developers' largest AI frustration is output that is almost right but not quite, with time-consuming AI-code debugging close behind [10]. OWASP's LLM Top 10 names prompt injection, sensitive information disclosure, supply-chain risk, improper output handling, and excessive agency as first-order AI application risks [11]. SetupBench and ContextBench add the operational failure modes: agents still struggle with environment bootstrap and useful context retrieval [12], [13].

Highest-priority controls. A humanlint-conformant repo should block merges when a changed path has no proof lane, a public boundary has handwritten contract mirrors, durable truth moves outside Rust/PostgreSQL ownership, security or secret scans are absent for risky changes, agent permissions are broader than the lane requires, user-facing UI lacks rendered proof, or `fallback`/`TODO`/`stub` ships without a scoped exception.

The taxonomy turns insult into audit surface. A finding is useful only when it names the rule, points at exact evidence, gives a bounded repair, and can be serialized into an agent queue. "This feels messy" is not actionable. `HLT-002-GENERATED-MUTATION` in `apps/web/src/generated/client.ts` with source `contracts/openapi/api.yaml` is actionable.

TABLE III
TECHNICAL PROMISE VERSUS STANDARD GRAVITY.

Language or ecosystem	Real promise	Adoption limiter	Agent-era penalty
Julia	Scientific productivity with performance-oriented multiple dispatch.	Broader product engineering ecosystem and deployment defaults lag dominant stacks.	Thinner general-product corpus and more local conventions.
D	Better C++-class systems ergonomics.	Could not displace entrenched C/C++ ecosystems and tooling gravity.	Smaller examples and weaker enterprise defaults.
Nim and Crystal	Python/Ruby-like ergonomics with native compilation.	Smaller communities, fewer boring production patterns.	Agents get fewer canonical examples and fewer tool-enforced boundaries.
Elm	Frontend reliability and no-runtime-exception ambition.	Ecosystem and interop constraints limited broad adoption.	Excellent lessons, but narrow hiring and corpus surface.
Reason/OCaml frontend	Typed functional UI with serious compiler roots.	Tooling and community fragmentation.	Harder for general agents to infer local conventions.
F# and Clojure	High-leverage functional design on mature runtimes.	Strong niches, smaller mainstream product footprint.	Expert-friendly abstractions can become opaque to repair agents.
Haskell and Scala	Powerful type-system and abstraction research translated to production niches.	Complexity and hiring/training friction.	Proof power helps only when conventions are explicit and common.
Elixir/Phoenix	Fault tolerance, supervision, realtime systems on the BEAM.	Specialist fit rather than universal product default.	Wins realtime collaboration, not general stack standardization.



Share	TLR bucket	Included faults
28%	Security	Generated-code flaws, secrets, prompt injection, excessive agency, supply chain.
20%	Business truth	False-green business logic, authorization drift, data isolation.
19%	Verification	False-green tests, pixel/UI, accessibility, proof quality.
17%	Contracts/data	Contract drift, wrong-layer persistence, generated mutation.
9%	Context/setup	Context retrieval, setup hallucination, instruction drift.
7%	Entropy	Dead language, mega files/functions, performance drift.

Fig. 1. TLR means Top-Level Risk. Shares are derived from the taxonomy's policy-weighted RPN buckets for humanlint 0.2.0; they are not incident-frequency measurements. The chart is intended to steer audit priority: security and business truth receive first repair attention even when easier style findings are more numerous.

VI. HUMANLINT STANDARD AND CONFORMANCE

humanlint is the control plane. It is not a general-purpose claim that all AI repositories should use one stack or one tool. It is a versioned standard for repositories that want to claim agent-native conformance. The standard binds prose, machine-readable maps, generated-zone policy, audit output, CI behavior, and repair evidence.

MUST, SHOULD, and MAY are used in the RFC sense inside the standard. A repository claiming HL3 or higher MUST provide a root agent router, owner map, test map, generated-zone manifest, version manifest, one-command fast lane, audit JSON, audit Markdown, and an ordered `agent_fix_queue`. It MUST NOT hand-edit generated zones, hide durable truth in the wrong layer, or merge high-risk changes without a mapped proof lane. It SHOULD provide SARIF output when the host CI can ingest it. SARIF is a planned output format for the audit, not a claim about the current v0.2 implementation.

Stable rule identifiers make the standard repairable. The initial rule family covers dead markers, generated-zone mutation, owner and proof routing, Python truth, wrong-layer DB access, handwritten contracts, false-green risk, generated-code security, secret sprawl, hostile context, overbroad agency, rendered UX, accessibility, setup/context gaps, supply-chain drift, opaque observability, and performance or concurrency drift. Appendix A expands these IDs into required evidence and repair fields.

Centerline drift is the score delta between a repository's claimed standard and its observed behavior. Hard caps are versioned policy, not final empirical truth. The point is to make governance explicit: teams can argue about weights and caps in releases, not in scattered PR comments.

VII. A 100-POINT STACK RUBRIC

The stack rubric scores durable product systems maintained by agents and humans together. It deliberately weights correctness, security, contracts, and repairability above first-draft velocity because AI already lowers the cost of plausible drafts. The scarce resource is trustworthy merge.

TABLE IV
RANKED VIBE-CODING FAULT TAXONOMY. RPN IS A HUMANLINT POLICY PRIORITY, NOT AN EMPIRICAL FREQUENCY CLAIM.

TLR	RPN	Fault	Why it happens	Required control
Business truth	125	False-green business logic	Code compiles and tests pass, but the product rule is wrong or asserted at the wrong level.	Domain invariants, property tests, golden workflows, DB constraints, incident replay tests.
Business truth	100	Authorization or data-isolation drift	Agent patches authz in UI, handler, or Python because it is locally easy.	Role matrix tests, negative authz tests, owner-map routing, RLS where useful.
Security	80	Plausible insecure generated code	Output looks idiomatic while hiding injection, XSS, unsafe deserialization, weak crypto, or logging flaws.	Security lane, SAST/SCA, secure-code rules, threat-model checklist, negative tests.
Security	80	Secrets and identity sprawl	Credentials leak through code, logs, prompts, fixtures, MCP/tool config, or copied examples.	Push protection, local/CI secret scans, redacted logs, credential inventory, transcript/artifact scan.
Security	75	Excessive agent agency	Broad terminal, browser, network, or filesystem permissions let agents take unsafe actions.	Least-privilege profiles, destructive-command gates, workspace-only writes, permission receipts.
Security	75	Prompt injection and hostile context	Issues, docs, pages, or comments ask the model to bypass trusted policy.	Source-trust hierarchy, untrusted-content isolation, tool-call validation, no secrets in context.
Verification	64	False-green tests	Agent writes tests that prove its implementation rather than the requirement.	Reviewer-owned acceptance criteria, mutation/fault checks, semantic assertions, no snapshot-only proof.
Context	64	Context retrieval failure	Agent over-reads, under-reads, or follows stale nearby patterns instead of the owner path.	Short root router, owner/test maps, local rules, symbol search, command-output filtering.
Contracts	60	Contract and DTO drift	Handwritten API types, local fetch wrappers, and stale clients become private contracts.	Generated clients only, contract drift lane, generated-zone lock, compatibility tests.
Contracts	60	Wrong-layer persistence	UI, API edge, scripts, or Python gain durable truth and bypass application/database policy.	DB access policy, adapters-only SQL, migration and constraint tests, forbidden imports.
Context	48	Environment/setup hallucination	Unclear setup leads agents to install random tools, change config, or skip DB/service proof.	One-command setup, pinned toolchain, devcontainer where useful, setup proof lane.
Verification	48	Pixel/UI regression	UI functions while layout, hierarchy, responsive state, overflow, or design tokens break.	Storybook states, Playwright screenshots, geometry rules, baseline governance, trace artifacts.
Verification	48	Accessibility regression	Visual output looks acceptable while labels, keyboard paths, focus, target size, or ARIA change.	axe/ARIA checks, keyboard journeys, focus/target geometry, expert review for semantic gaps.
Security	45	Dependency and supply-chain drift	Agent adds convenient packages without license, vulnerability, provenance, or lockfile review.	Dependency rationale, lockfile diff review, OSV/SCA, SBOM, SLSA/provenance checks.
Repair	45	Observability and repair opacity	Generic errors and print logs leave no stable path from failure to fix.	Problem details, stable error codes, OpenTelemetry attributes, correlation IDs, repair hints.
Entropy	45	Dead-language normalization	`temporary`, `fallback`, `stub`, `TODO`, and `legacy` become permanent states.	Banned marker scan, scoped allowlist, owner, expiry, exception record.
Entropy	36	Performance, memory, or concurrency regression	Plausible code duplicates queries, polls, leaks memory, omits cancellation, or ignores backpressure.	Benchmarks, query-plan checks, load smoke tests, allocation budgets, tracing.
Entropy	36	Mega-file or mega-function sprawl	Repair locality disappears and agents edit broad surfaces.	LOC/function caps, split by owner/use case, focused tests before behavior edits.
Contracts	30	Generated-zone mutation	Generated output becomes forked source truth.	Generated-zone manifest, source contract change, regeneration command, drift check.
Context	30	Documentation or instruction drift	Tool-specific files contradict canonical policy and agents choose randomly.	Canonical manifest, generated mirrors, instruction lint, short root guidance.

Points are awarded only for structural properties. "We review carefully" is not a security control. "The senior engineer knows where SQL belongs" is not a boundary. "The README explains it" is not a generated contract. The stack must make the wrong path hard to express and the right path cheap to prove.

Hard filters apply before points. A stack with no deterministic local proof lane, no security lane for high-risk changes, no generated contract discipline, or no credible data boundary cannot score as agent-native even if it is pleasant to write. This is why the paper treats caps, uncertainty, and sensitivity as part of the result rather than footnotes.

VIII. STACK RANKING, SENSITIVITY, AND EXCEPTIONS

The top-five result is a default recommendation, not a metaphysical law. The scores assume durable product systems with web product surface, API/service behavior, operational security requirements, and long-running maintenance by agents and humans together.

Sensitivity is concentrated in three weights: memory safety/security, ecosystem corpus strength, and runtime/concurrency. Increase ecosystem weight and Go, .NET, and JVM improve. Increase type-state and memory-safety weight and Rust separates. Increase product velocity and full-stack TypeScript rises, but it hits hard filters unless durable truth, generated contracts, and server ownership are explicit.

Kafka deserves a separate note because it is both strong and not the standard. It remains a serious event-streaming contender in brownfield systems: its semantics, ecosystem, and operational familiarity are real. But JVM-bound streaming infrastructure carries exactly the runtime surface and migration gravity this paper is trying to shrink. The humanlint position

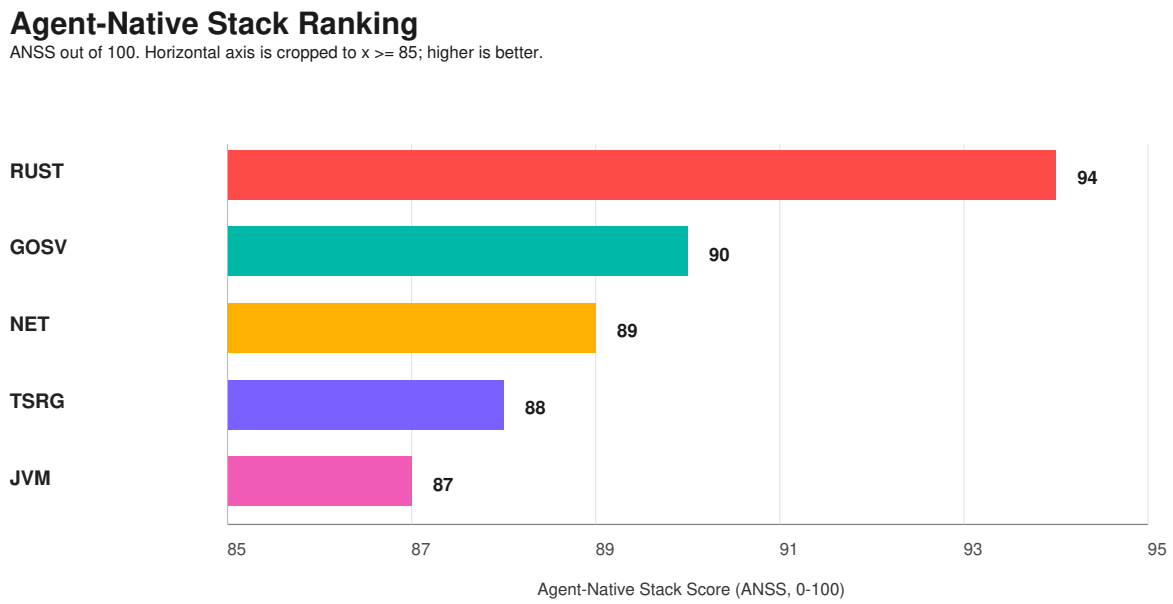


Fig. 2. Top-five stack ranking by Agent-Native Stack Score (ANSS), the weighted humanlint 100-point rubric, with the x-axis starting at 85. Labels: RUST = Rust core, GOSV = Go services, NET = C#/.NET, TSRG = TypeScript product plane with Rust/Go compute cells, and JVM = Kotlin/Java. All entries assume TypeScript/React/Vite product surface and PostgreSQL durable truth; streaming buses are workload-specific additions.

TABLE V
HUMANLINT CONFORMANCE LEVELS.

Level	Meaning
HL0	Unscored: no reliable root instructions or proof lane.
HL1	Advisory: audit runs and emits JSON/Markdown, but findings do not block merge.
HL2	Guarded: critical caps, generated-zone mutation, and missing fast lane block merge.
HL3	Standard: high and critical findings block merge; score floor and conformance metadata are enforced.
HL4	Ratchet: score cannot regress without an exception; repair queue is tracked to closure.
HL5	Release contract: audit, security, contracts, DB, e2e, and artifact versions gate release.

is explicit: use Kafka when the system already needs it, treat it as necessary-evil workload infrastructure rather than stack identity, and expect agent-native systems work to push toward a Kafka-class Rust-native replacement as AI compresses implementation cost.

IX. Default Winner Architecture

Once the standard identifies a default, continuing to optimize for every runner-up becomes noise. The agent needs one default map. The audit needs one default policy. CI needs one default set of gates. Exceptions remain available, but they must be named and versioned.

The filesystem should be a routing table. A patch touching a payment invariant should land in Rust domain/application and PostgreSQL constraints. A patch touching a form should land in TypeScript UI and generated client contracts. A patch touching model behavior should land in the bounded Python service and contract tests. If the agent must infer ownership from convention, the repo has already failed.

X. Agent Repository Controls and Tool Adapters

Public agent tooling converges on one lesson: instructions matter, but deterministic control surfaces matter more. Codex supports project guidance through `AGENTS.md` [24]. The community `AGENTS.md` specification frames the file as a plain Markdown companion for coding agents [25]. Claude Code loads project memory through `CLAUDE.md` and related hierarchy [26]. Cursor uses project rules with scope metadata [27]. GitHub Copilot supports repository and path-specific custom instructions [28]. These mechanisms should not become divergent manuals. They should be generated or mirrored from one canonical standard.

Token minimization is not a prompting trick; it is repository design. Large root instructions, duplicate architecture documents, and noisy command output waste context and invite contradiction. ContextBench and SWE-Effi support this direction by treating useful context and resource constraints as first-class bottlenecks [13], [29]. RTK-style command filtering is one implementation of the larger principle: show the agent the evidence it needs, not every byte the tool can emit.

XI. Continuous Proof, CI, and Test Explosion

Agent-native testing should be organized by proof question, not author convenience. Domain invariants need Rust unit and property tests. Application workflows need authorization, idempotency, and transaction tests. Adapters need integration and contract tests. PostgreSQL needs migration, constraint,

TABLE VI
HUMANLINT 100-POINT STACK RUBRIC FOR AGENT-NATIVE PRODUCT ENGINEERING.

Criterion	Weight	Full-credit signal	Evidence and argument
Agent-verifiable correctness loop	18	Fast type/compile checks, deterministic test routing, generated contracts, property tests, reproducible local and CI lanes.	AI output must be treated as suspect until a proof loop rejects or accepts it; setup and context benchmarks show environment and retrieval reliability are central bottlenecks [12], [13].
Security, memory safety, supply chain	16	Memory-safe core, secret scanning, SCA, SBOM/provenance, lockfiles, unsafe ledger, dependency rationale.	CISA/NSA memory-safety guidance, Veracode GenAI findings, and GitGuardian secrets evidence all push security into structural gates [3], [4], [7].
Runtime performance, memory, cloud cost	13	Efficient hot path, predictable latency, bounded memory, cheap concurrency, low cold-start cost.	Agent-era systems are compute-heavy and parallel; runtime cost becomes architectural, not cosmetic.
Concurrency and resilience	12	Structured async, cancellation, backpressure, idempotency, retries, dead-letter handling, replayable workflows.	Go survey evidence shows broad AI use but persistent quality concerns; concurrency needs guardrails rather than cleverness [14].
Boundary and contract integrity	11	OpenAPI/Protobuf/JSON Schema sources, generated clients, schema drift checks, migration gates.	Cross-language systems work only when contracts are generated and tested; OpenAPI Generator and Buf provide mature paths [15], [16].
Simplicity and review surface	10	Small files, narrow modules, explicit ownership, boring dependency direction, low duplication.	Agents over-edit when boundaries are implicit; SWE-agent and SWE-bench show interfaces and environment shape affect agent results [17], [18].
Ecosystem, hiring, AI corpus strength	8	Large corpus, maintained packages, common deployment patterns, strong model familiarity.	GitHub Octoverse language patterns and TypeScript's rise matter because public examples are part of the agent substrate [1].
Observability, auditability, provenance	6	OpenTelemetry traces/metrics/logs, request IDs, release provenance, repair receipts.	OpenTelemetry gives a vendor-neutral vocabulary for post-merge repair [19], [20].
Data integrity and durable workflow fit	4	PostgreSQL constraints, migrations, indexes, RLS where useful, transactional workflow safety.	PostgreSQL constraints and row-security policies move durable truth below fallible application code [21], [22].
Product velocity and interop	2	Fast UI loop, generated clients, easy local dev, standard browser ecosystem.	TypeScript 7 and Vite 8 improve feedback speed, but velocity is deliberately weighted below proof [6], [23].

TABLE VII
TOP-FIVE STACK RESULTS UNDER THE HUMANLINT RUBRIC.

Rank	Stack	ANSS
1	Rust core + TypeScript/React/Vite + PostgreSQL	94
2	Go services + TypeScript/React/Vite + PostgreSQL	90
3	C#/.NET + TypeScript/React/Vite + PostgreSQL	89
4	TypeScript product plane + Rust/Go compute cells + PostgreSQL	88
5	Kotlin/Java JVM + TypeScript/React/Vite + PostgreSQL	87

and schema drift checks. The browser needs component tests, rendered UX proof, and Playwright coverage for critical paths. Public benchmarks and open-source agents show that setup reliability, context retrieval, and interface design are central bottlenecks for autonomous repair [12], [13], [17], [18], [30], [31].

The explosion of tests is useful only if routed. A repo with thousands of tests and no `test-map.json` still asks agents to guess. The standard therefore requires a fast lane for changed files, a contract lane for generated interfaces, a security lane for dependency and secret risk, a DB lane for migrations, a web lane for component and rendered UX proof, an e2e lane for critical browser behavior, and a release lane for full confidence.

CI modes should be explicit. *Advisory* mode emits reports but does not fail merge. *Guarded* mode blocks critical caps. *Standard* mode enforces high and critical findings plus a score floor. *Ratchet* mode prevents score regression without a named exception. *Release* mode gates shipped artifacts on audit, contracts, security, DB, e2e, and version checks.

XII. Rendered UX and Browser-Step QA

The browser is a real runtime, not a screenshot afterthought. Agents can create UI that compiles, passes component tests, and still fails through overlapping text, broken responsive layout, invisible controls, stale selectors, missing focus states, blank canvases, cramped buttons, clipped labels, sticky footer collisions, or layout shift. Playwright's official guidance emphasizes user-visible behavior, resilient locators, isolation, and

TABLE VIII
REPRESENTATIVE SCORE DECOMPOSITION. BANDS EXPRESS POLICY UNCERTAINTY, NOT STATISTICAL CONFIDENCE.

Stack family	Proof	Sec.	Runtime	Boundary	Ops	Band	Exception logic
Rust + TS/Vite + Postgres	18	16	13	11	36	±3	Default when no stronger domain constraint exists.
Go + TS/Vite + Postgres	16	13	13	10	38	±4	Strong cloud-service default where simplicity beats type-state power.
.NET + TS/Vite + Postgres	16	13	12	10	38	±4	Strong regulated enterprise answer when Microsoft platform fit speeds proof.
Full-stack Next.js + Postgres	13	10	9	8	36	±6	Excellent product loop; needs stricter backend/data partitioning to avoid truth drift.
Python/FastAPI + Postgres	10	8	8	7	34	±7	Useful AI/data boundary; risky as durable product core.
Rails + Postgres	11	9	8	7	36	±6	Fast CRUD app fit; needs generated contracts and stronger service boundaries.
Elixir/Phoenix + Postgres	14	11	11	8	37	±5	Specialist override for realtime collaboration and supervision-heavy systems.
JVM/Spring or Kotlin + Postgres	15	11	11	10	40	±5	Brownfield and enterprise fit can outweigh default-stack migration cost.

```
repo/
  apps/
    web/              TS/React/Vite UI
    api/              Rust HTTP/RPC edge
  crates/
    domain/           pure invariants
    application/      use cases/authz/tx
    adapters/         Postgres/queues/APIs
    workers/          jobs/replay/backoff
  contracts/          OpenAPI/Protobuf/JSON
  db/                 migrations/constraints
  python/ai-service/  model/eval boundary
  ops/                CI/security/telemetry
```

```
repo/
  AGENTS.md           short root router
  agent/
    owner-map.json    path -> owner
    test-map.json     path -> proof lane
    proof-lanes.toml  runnable commands
    generated-zones.toml
    standard-version.toml
    repo-score.json
  docs/
    decisions/
    exceptions/
    repair/
    runbooks/
```

Fig. 3. Ownership spine and agent-control spine for the default stack.

web-first assertions, which fit agent repair better than brittle sleeps and CSS selectors [32], [33].

The human QA gap is rendered experience. A test can prove that a button exists and calls the correct API; it usually cannot prove that the button is too close to the viewport edge or that a German label breaks a card at 375px. A screenshot diff can prove that pixels changed, but not whether the change is intentional, harmful, or merely anti-aliasing noise. The standard therefore treats rendered UX as a layered proof system: design-system constraints, generated contracts, deterministic Storybook states, Playwright screenshots, DOM geometry rules, accessibility rules, visual regression, AI/CV triage, and human feedback labels.

Open-source tools already solve real pieces of this. Storybook can turn every story into a visual baseline [34]. Playwright can compare screenshots with `toHaveScreenshot()` and warns that rendering environments must be stable for reliable baselines [35]. Argos and BackstopJS provide visual review and screenshot-diff workflows [36], [37]. Playwright can run axe-based

TABLE IX
DEFAULT OWNERSHIP CELLS FOR THE WINNING STACK.

Cell	Owns	Must not own
`apps/web`	React UI, route state, forms, local validation, generated API clients, Playwright tests.	Secrets, durable truth, direct SQL, core authorization, handwritten API mirrors.
`apps/api`	Rust HTTP/RPC edge, request normalization, auth middleware, idempotency keys, response shaping.	Domain invariants hidden in handlers, scattered SQL, UI concerns.
`domain` crate	IDs, value objects, invariants, state machines, pure decisions, stable error types.	I/O, env reads, network, filesystem, database clients, framework code.
`application` crate	Commands, authz, transaction orchestration, idempotency, workflow use cases.	Transport details, SQL drivers, UI state, prompt/model logic.
`adapters` crate	PostgreSQL access, queues, external APIs, filesystem, environment, provider clients.	Domain rules or authorization policy.
`workers` crate	Async jobs, retries, backoff, workflow replay, scheduled tasks.	Duplicated invariants or unowned product truth.
`contracts`	OpenAPI, Protobuf, JSON Schema, generated stubs and clients.	Hand-edited generated outputs or business logic.
`db`	Migrations, constraints, indexes, RLS, seed rules, schema snapshots.	Application orchestration or hidden business process.
`python/ai-service`	Inference, embeddings, evals, prompt experiments, offline transforms, typed AI API.	Product truth, authz, billing, direct production DB ownership.
`ops`	CI, release, telemetry, secret scanning, SBOM/SCA, provenance, policy gates.	Feature behavior hidden behind manual operations.

TABLE X
AGENT TOOL ADAPTER MATRIX.

Tool family	Native surface	humanlint adapter	Non-negotiable rule
Codex	`AGENTS.md`, local scoped instructions.	Root router plus local ownership files and mapped proof commands.	Do not duplicate durable policy outside the standard.
Claude Code	`CLAUDE.md`, project memory, local rules.	Mirror root router, version, commands, and generated zones.	Memory may summarize policy but not grant broader permissions.
Cursor	`.cursor/rules` with scope metadata.	Generate scoped rules from owner map and test map.	Glob rules must agree with owner-map paths.
GitHub Copilot	Repository and path-specific custom instructions.	Short repo-wide mirror plus path instructions for high-risk owners.	No private architecture fork in IDE guidance.
Antigravity-style IDEs	Mission-control UI, terminal/browser approvals, local tasks.	Versioned approval checklist and proof-lane mapping.	Approval policy must respect generated zones and secrets.
Terminal agents	CLI prompt files, repo maps, shell wrappers.	Root router, filtered command output, raw-output escape hatch.	Tool convenience must not bypass CI gates.

accessibility checks, while its docs warn that automation cannot find every accessibility issue [38]. MSW lets Storybook stories use deterministic network responses rather than live API luck [39]. The right open-source posture is composition: use those tools for capture, baselines, a11y, mocks, and review, while humanlint owns the policy layer they do not share. The missing core library is a repo-local geometry oracle: a deterministic rule set over DOM boxes, computed styles, accessibility names, scroll sizes, focus state, and viewport state.

The feedback loop must also be structured. A human approval should label whether a delta is a true layout bug, intentional visual change, acceptable variance, design-system violation, accessibility defect, copy/content issue, responsive-only issue, browser-only issue, false positive, or product/design review item. Storing those labels with the route, viewport, screenshot crop, DOM selector, rule ID, owner, and expiration turns taste review into training data for future thresholds and exceptions.

Some criteria are already standards-backed. WCAG 2.2 target-size guidance defines a 24 by 24 CSS-pixel minimum or sufficient spacing exceptions for pointer targets [40]. Core Web Vitals guidance recommends CLS of 0.1 or less for a good visual-stability experience [41]. Humanlint should therefore treat target-size failures, clipped required text, horizontal overflow, sticky obstruction, missing focus visibility, severe CLS, and unapproved token violations as hard evidence, not as taste debates.

Pixel QA is first-class but not exhaustive on every pull request. The standard risk-routes it. Critical flows, high-risk UI surfaces, visual diff baselines, payments, auth, admin actions, onboarding, and canvas/3D surfaces need browser-step proof in the PR lane. Full viewport and device matrices belong in nightly or release lanes unless the changed path directly

TABLE XI
MINIMUM PROOF LANES BY OWNERSHIP CELL.

Layer	Minimum proof
Rust domain	Unit tests plus property tests for invariants and state machines.
Rust application	Integration tests for authz, idempotency, transactions, workflows.
Rust adapters	DB/external integration tests and failure injection.
Contracts	Generation check, drift check, compatibility check.
PostgreSQL	Migration apply, constraint tests, RLS policy tests where used.
TypeScript web	Typecheck, component tests, rendered UX QA, Playwright critical paths.
Python AI service	Contract tests, eval suites, no direct product-truth ownership.
Ops/security	Secret scan, dependency scan, provenance, audit gate.

touches those surfaces.

A repair receipt for UI should include the route, viewport, action sequence, screenshot or trace path, assertion, and changed files. This turns visual QA from "a human clicked around" into evidence a future agent can replay.

XIII. Security, Supply Chain, and Permissions

Agentic coding expands the write surface. The security posture must therefore be structural. CISA/NSA guidance pushes memory-safe languages as a way to reduce memory-related vulnerability classes [7]. Veracode's GenAI security results show syntax success does not imply secure code [3]. GitGuardian's secret-sprawl evidence shows AI-churned repositories need stronger secret gates, not more trust [4].

Permission policy should be boring. Agents should not receive broad filesystem, network, or credential access merely because a task is urgent. High-risk actions need named lanes and evidence: dependency install, secret material, generated-zone writes, migrations, release artifacts, and browser automation against privileged surfaces. A good permission system makes the safe path fast and the dangerous path visible.

XIV. Exceptions, Observability, and Repair Receipts

Exceptions are where teams should pool hard-earned repair knowledge. A normal exception tells a human that something failed. An agent-friendly exception tells a repair system what invariant was protected, why it failed, where to read the rule, and what repairs are usually legal.

The shape should combine language-native errors with standardized machine-readable fields. HTTP APIs should use RFC 9457 problem details for boundary errors [45]. Observability should emit OpenTelemetry exception attributes such as exception type, message, stack trace, and error type [20], [46]. TypeScript can preserve underlying causes with `Error.cause` [47]. Rust should prefer typed errors for domain/application code, using context wrappers only at boundaries where opaque provider detail is acceptable [48], [49].

Every controlled error that can reach logs, API responses, background jobs, or tests should expose a stable name, stable code, purpose, reason, common fixes, documentation URL, safe user message, correlation ID, and source cause when safe. The point is not verbosity. The point is to make a failing test or production trace directly actionable for the next agent.

XV. Migration, Versioning, and Governance

Teams do not reach the default stack through rewrite fantasy. They move by shrinking ambiguity. First add the audit in advisory mode. Then add root instructions, owner maps, test maps, generated zones, version manifests, and proof lanes. Then generate contracts, isolate database access, move durable truth into PostgreSQL constraints and Rust application/domain code, box Python into AI/data boundaries, and add Playwright coverage for critical product paths. Only after the repo can route and prove changes should teams accelerate agent-driven migration.

The standard must version quickly. The paper, audit script, agent artifacts, standard documents, generated artifacts, and CI policy should share a manifest. In this repo, `agent/standard-version.toml` binds `paper/humanlint.tex`, `paper/humanlint.pdf`, `paper/humanlint.md`, `docs/agent-native-standard.md`, and `agent/HUMANLINT_STANDARD.md`. TeX remains the canonical paper source. The Markdown paper is an agent-friendly companion, not the source of truth.

Governance should separate empirical claims from policy choices. Scoring weights, hard caps, conformance levels, and rule IDs are versioned policy. Public source claims require citations. New audit output fields require schema versioning. Breaking compliance changes require migration notes and example repairs. The goal is not to freeze one 2026 opinion forever; it is to make the standard itself updatable without letting every repository invent a private dialect.

XVI. Limitations and Research Agenda

The scoring weights need empirical calibration across real repositories. Agent-friendly exception patterns need comparative studies across Rust, TypeScript, and Python. The file-size limits should be tested against repair accuracy, not taste. Tool-specific instruction behavior changes quickly. Public benchmarks still struggle to measure long-running maintenance, security regressions, UI correctness, and production repair quality. Those gaps are a research agenda, not reasons to keep shipping ambiguous repos.

The strongest version of the criticism is useful: a standard can become ceremony, and a stack recommendation can become dogma. humanlint should be judged by whether it improves repair locality, reduces unsafe merges, shortens review cycles, and produces better evidence. If a team can prove those outcomes with a different stack and a compatible control plane, the exception should be documented and studied.

TABLE XII
RENDERED UX QA DETECTORS FOR THE TYPESCRIPT PRODUCT SURFACE.

Detector	Machine signal	Merge behavior
Design-system constraints	Component imports, token use, no raw one-off spacing/color/z-index.	Block unapproved design-language drift.
Storybook state coverage	Normal, loading, empty, error, long text, locale, theme, mobile, and role states.	Require stories for reusable components and key screens.
Playwright screenshots	Component and flow screenshots under pinned browser, font, viewport, timezone, and locale settings.	Require review for protected baselines.
DOM geometry rules	Bounding boxes, computed styles, scroll sizes, target spacing, overlap, clipping, wrapping, nested scrollbars, and fixed/sticky collisions.	Block objective layout failures before human QA.
Accessibility checks	axe/WCAG checks, keyboard paths, focus visibility, form labels, ARIA names, and target size.	Block critical known violations; route ambiguous cases to expert review.
Layout stability	CLS and late-shift evidence from Lighthouse or Web Vitals.	Block severe movement on product-critical pages.
AI/CV triage	Semantic review of ambiguous visual deltas and screenshot crops.	Warn or route to humans; do not replace deterministic gates.
Human feedback loop	Labels for true defect, intentional change, false positive, acceptable variance, missing rule, or design-system bug.	Improve thresholds, baselines, stories, and future rules.

TABLE XIII
BROWSER-STEP QA ROUTING.

Risk	Required proof
Critical user journey	Playwright path with role/text/test-id locators and trace artifact.
Responsive layout	Desktop and mobile screenshots with overlap and clipping checks.
Canvas, map, video, 3D	Pixel nonblank check plus interaction or animation evidence.
Design-system component	Component tests plus one rendered browser state per variant class.
Low-risk copy change	Component or lint proof; full matrix deferred unless release lane catches drift.

XVII. CONCLUSION

Agent-native engineering is an adaptation problem. Teams that optimize for human comfort alone will get faster at creating ambiguity. Teams that optimize for verification will get faster at rejecting bad code, localizing mistakes, repairing real failures, and changing architecture without guessing.

The winner is not the stack that flatters the author. The winner is the repo-and-stack system that makes invalid states hard to express, boundary drift hard to hide, security failures hard to merge, production behavior easy to trace, and repair work easy to assign. The default target is Rust core, TypeScript/React/Vite product surface, PostgreSQL truth, generated contracts, and bounded Python. The repo around it must become a verification interface. Anything less is just faster vibe coding.

TABLE XIV
SECURITY CONTROLS FOR AGENT-NATIVE REPOSITORIES.

Risk	Control	Repair evidence
Prompt injection and hostile context	Source hierarchy: root instructions, local owner rules, trusted docs, then untrusted issue/web text.	Receipt names ignored instruction, trusted rule, and resulting safe action.
Secrets and credentials	Secret scanning, no secrets in UI or docs, redacted logs, no model prompt leakage.	Scanner artifact, redaction test, finding closure.
Dependency sprawl	Lockfiles, SCA, SBOM/provenance, dependency rationale for new packages.	Package diff, risk reason, scanner result [42]–[44].
Sandbox escape or tool overreach	Least-privilege tools, explicit write scope, generated-zone protections.	Changed-path report and policy decision.
Unsafe code and FFI	Rust unsafe ledger, boundary tests, memory-safety review.	Ledger entry, invariant tests, owner approval.
False-green security	Security lane mapped by path and risk, not by agent choice.	CI job link and report artifact.

TABLE XV
MINIMUM REPAIR RECEIPT FIELDS.

Field	Purpose
`rule_id`	Stable humanlint rule or exception ID.
`owner`	Owning path, team, or layer.
`failure`	What invariant or proof lane failed.
`changed_paths`	Exact edit surface.
`proof_lane`	Command or CI job that accepted the repair.
artifact versions	Standard, auditor, schema, paper, and generated artifact bindings.
`next_repair`	Bounded follow-up when the fix is partial.

TABLE XVI
CI ADOPTION MODES.

Mode	Merge behavior
Advisory	Emit JSON/Markdown reports and never block.
Guarded	Block critical caps and malformed output.
Standard	Block high and critical findings plus score-floor failure.
Ratchet	Prevent score regression without a dated exception.
Release	Gate shipped artifacts on audit, tests, security, contracts, DB, e2e, and versions.

APPENDIX A
RULE IDS AND CONFORMANCE EVIDENCE

Stable rule identifiers make audit output durable enough for repair agents, CI annotations, and release notes.

TABLE XVII
INITIAL HUMANLINT RULE IDENTIFIERS.

Rule ID	Required evidence	Default repair
HLT-001-DEAD-MARKER	Path, line, matched term, allowlist decision.	Rename to the real state, implement it, delete it, or create a dated exception.
HLT-002-GENERATED-MUTATION	Generated path, source path, regeneration command, diff context.	Change the source contract or generator and regenerate.
HLT-003-OWNERLESS-PATH	Changed path with no owner-map match.	Add owner-map entry or move code into owned path.
HLT-004-UNMAPPED-PROOF	Changed path with no test-map lane.	Add or select deterministic proof command.
HLT-005-PYTHON-PRODUCT-TRUTH	Python path owning product truth, authz, or production DB writes.	Move durable behavior to Rust/PostgreSQL or box Python as AI/data service.
HLT-006-DIRECT-DB-WRONG-LAYER	SQL or DB client marker in UI/domain/application/Python truth path.	Move data access to adapters or migrations.
HLT-007-HANDWRITTEN-CONTRACT	Mirrored DTO, local fetch wrapper, or hand-authored client drift.	Generate from OpenAPI/Protobuf/JSON Schema.
HLT-008-FALSE-GREEN-RISK	Changed behavior accepted by unrelated, weak, skipped, or mock-only lane.	Route to owner lane and add semantic assertion.
HLT-009-GENERATED-SECURITY	AI-generated code changes auth, input handling, crypto, deserialization, logging, or filesystem behavior without security proof.	Run security lane, add negative tests, and attach threat-model evidence.
HLT-010-SECRET-SPRAWL	Secret-like value, broad env dump, unsafe fixture, prompt transcript leak, or missing secret scan.	Remove secret, rotate if needed, add scanner evidence and redaction tests.
HLT-011-PROMPT-INJECTION	Untrusted issue, web, doc, or tool output changes trusted policy or tool behavior.	Preserve source hierarchy, isolate untrusted context, and record ignored instruction.
HLT-012-OVERBROAD-AGENCY	Agent/tool lane allows broad shell, browser, network, migration, delete, or credential actions.	Add least-privilege permission profile and approval gate.
HLT-013-RENDERED-UX-GAP	User-facing UI change lacks screenshot, trace, geometry, baseline, or viewport proof.	Add Storybook/Playwright/geometry evidence for changed surface.
HLT-014-A11Y-GAP	UI lacks labels, keyboard path, focus visibility, target size, ARIA, or axe evidence.	Add accessibility checks and route semantic gaps to expert review.
HLT-015-CONTEXT-SETUP-GAP	Repo lacks deterministic setup/context routing or agent follows stale surrounding context.	Add one-command setup, owner/test maps, local rules, and filtered output.
HLT-016-SUPPLY-CHAIN-DRIFT	Dependency added without rationale, lockfile review, vulnerability scan, SBOM, or provenance.	Add dependency rationale, scanner result, and provenance evidence.
HLT-017-OPAQUE-OBSERVABILITY	Boundary failure lacks stable code, trace/correlation ID, safe message, or repair hint.	Use problem details, OpenTelemetry attributes, and agent-friendly errors.
HLT-018-PERF-CONCURRENCY-DRIFT	Patch adds polling, query duplication, unbounded concurrency, missing cancellation, or memory risk.	Add benchmark/query/load/backpressure evidence and trace the hot path.

The audit JSON should include the rule ID whenever a finding maps cleanly to one of these rules. Rules outside the initial set may use category IDs until they stabilize.

```
{
  "standard_version": "0.2.0",
  "auditor_version": "0.2.0",
  "schema_version": "1.0.0",
  "paper_edition": "2026.05-ed1",
  "target_stack_id": "rust-ts-vite-react-postgres-bounded-python",
  "agent_fix_queue": [
    {
      "rule_id": "HLT-002-GENERATED-MUTATION",
      "path": "apps/web/src/generated/client.ts",
      "priority": "high",
      "task": "change contracts/openapi/api.yaml and regenerate"
    }
  ]
}
```

APPENDIX B
VERSIONED ARTIFACT MANIFEST

The canonical manifest is `agent/standard-version.toml`. It binds the paper source, rendered paper, agent Markdown companion, full coding standard, and agent standard brief.

```
standard = "humanlint"
standard_version = "0.2.0"
paper_edition = "2026.05-ed1"
auditor_version = "0.2.0"
schema_version = "1.0.0"
target_stack = "rust-ts-vite-react-postgres-bounded-python"
published = "2026-05-01"
```

```
[[artifact]]
id = "paper-source"
path = "paper/humanlint.tex"
kind = "tex-wrapper"
version_field = "paper_edition"
source = "paper/tex/"
command = "just paper"

[[artifact]]
id = "paper-render"
path = "paper/humanlint.pdf"
kind = "pdf"
version_field = "paper_edition"
source = "paper/humanlint.tex"
command = "just paper"
generated = true

[[artifact]]
id = "paper-agent-md"
path = "paper/humanlint.md"
kind = "agent-markdown"
version_field = "paper_edition"

[[artifact]]
id = "coding-standard"
path = "docs/agent-native-standard.md"
kind = "standard"
version_field = "standard_version"

[[artifact]]
id = "agent-standard-brief"
path = "agent/HUMANLINT_STANDARD.md"
kind = "agent-brief"
version_field = "standard_version"
```

`tools/check_versions.py` verifies that `VERSION`, `pyproject.toml`, package metadata, CLI constants, standard documents, companion Markdown, artifact bindings, and generated-zone registration agree.

APPENDIX C
EXCEPTION AND REPAIR TEMPLATES

Exceptions are allowed only when they are named, scoped, owned, testable, and temporary enough for an agent to repair later.

```
docs/exceptions/<ID>.md

# <ID>: <short name>

Owner: <team or owner cell>
Scope: <exact files, crates, services, contracts, or zones>
Purpose: <what this exception permits>
Reason: <why the normal standard cannot be followed now>
Expiration: <date, milestone, or measurable exit condition>
Allowed edits: <what agents may change without widening scope>
Forbidden edits: <what agents must not touch>
Required proof: <test-map lane names and commands>
Common fixes:
  - <first bounded repair pattern>
  - <second bounded repair pattern>
Audit behavior: <allowed term/path or score cap policy>
```

```
docs/repair/<YYYY-MM-DD>-<short-id>.md

Rule: <HLT rule id>
Owner: <owner cell>
Failure: <what failed and where>
Changed paths:
  - <path>
Proof lane: <command or CI job>
Artifacts:
  - repo-score.json
  - trace.zip
  - screenshot.png
Result: <accepted, rejected, partial>
Next repair: <bounded follow-up or none>
```

APPENDIX D
CANONICAL FILE TREE DIAGRAMS

The body uses short diagrams so the reader can see the intended shape without leaving the ranking argument. The expanded diagrams below are the normative target shape for a default humanlint repository. Names may vary, but ownership, proof lanes, generated zones, durable truth, and repair evidence should remain explicit.

```
repo/
  apps/
    web/                        TypeScript/React/Vite UI
      src/
        app/                    routes, shells, providers
        components/             design-system composition only
        generated/              API clients and typed contracts
        stories/                Storybook state matrix
        test/                   Vitest, Testing Library, fixtures
      playwright/               critical flows and screenshots
      storybook/                rendered component states
    api/                        Rust HTTP/RPC edge
      src/
        routes/                 request decode and response encode
        generated/              generated protocol bindings
        auth/                   session bridge, not domain policy
  crates/
    domain/                     pure invariants and policy
    application/                use cases, authz, transactions
    adapters/
      postgres/                 SQLx/queries, migrations, repositories
      queues/                   jobs, replay, backoff
      external/                 third-party clients
    workers/                    async jobs and projections
  contracts/
    openapi/                    OpenAPI source
    proto/                      protobuf source
    jsonschema/                 emitted JSON Schema when needed
  db/
    migrations/                 durable schema history
    seeds/                      deterministic test scenarios
  python/ai-service/            bounded AI/data boundary only
  packages/
    ux-qa/                      rendered UX geometry runtime
  ops/
    ci/
    security/
    telemetry/
```

```
repo/
  AGENTS.md                     short root router
  agent/
    HUMANLINT_STANDARD.md       brief agent bootstrap
    owner-map.json              path -> accountable owner
    test-map.json               path -> proof lane
    proof-lanes.toml            runnable validation commands
    generated-zones.toml        generated file policy
    standard-version.toml       versioned artifact manifest
    audit-policy.toml           local scoring and exceptions
  docs/
    agent-native-standard.md    full standard
    mission.md                  paper mission and scope
    testing.md                  lane doctrine
    audit-rubric.md             score and cap semantics
    decisions/                  architecture decision records
    exceptions/                 named temporary deviations
    repair/                     repair receipts and artifacts
    runbooks/                   operational repair procedures
  .github/
    workflows/
      humanlint-audit.yml       audit, UX QA, paper lanes
  repo-score.json               generated audit output
  repo-score.md                 generated review surface
```

REFERENCES

[1] GitHub, "Octoverse: A new developer joins github every second as ai leads typescript to #1," https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/, 2025, gitHub Blog, published 2025; accessed 2026-05-01.

[2] DORA, "State of ai-assisted software development 2025," https://dora.dev/research/2025/dora-report/, 2025, dORA research page, accessed 2026-05-01.

[3] Veracode, "2025 genai code security report," https://www.veracode.com/wp-content/uploads/2025_GenAI_Code_Security_Report_Final.pdf, 2025, official report PDF, accessed 2026-05-01.

[4] GitGuardian, "The state of secrets sprawl report 2026," https://www.gitguardian.com/state-of-secrets-sprawl-report-2026, 2026, annual report page, accessed 2026-05-01.

[5] ——, "The state of secrets sprawl 2026: Ai-service leaks surge 81% and 29m secrets hit public github," https://blog.gitguardian.com/the-state-of-secrets-sprawl-2026/, 2026, official companion blog summary, published 2026-03-17; accessed 2026-05-01.

[6] Microsoft, "Announcing typescript 7.0 beta," https://devblogs.microsoft.com/typescript/announcing-typescript-7-0-beta/, 2026, typeScript Dev Blog, published 2026-04-21; accessed 2026-05-01.

[7] National Security Agency (NSA) and Cybersecurity and Infrastructure Security Agency (CISA), "Memory safe languages: Reducing vulnerabilities in modern software development," https://www.cisa.gov/resources-tools/resources/memory-safe-languages-reducing-vulnerabilities-modern-software-development, 2025, nSA/CISA cybersecurity information sheet, June 2025; accessed 2026-05-01.

[8] Python Software Foundation, "Python release python 3.14.0," https://www.python.org/downloads/release/python-3140/, 2025, release date Oct. 7, 2025; accessed 2026-05-01.

[9] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, "Julia: A fresh approach to numerical computing," *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017.

[10] Stack Overflow, "Ai: 2025 developer survey," https://survey.stackoverflow.co/2025/ai, 2025, official developer survey AI section; accessed 2026-05-01.

[11] OWASP Foundation, "Owasp top 10 for large language model applications v2025," https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-v2025.pdf, 2025, official OWASP project PDF; accessed 2026-05-01.

[12] SetupBench Authors, "Setupbench: Assessing software engineering agents' ability to bootstrap development environments," *arXiv preprint arXiv:2507.09063*, 2025.

[13] ContextBench Authors, "Contextbench: A benchmark for context retrieval in coding agents," *arXiv preprint arXiv:2602.05892*, 2026.

[14] The Go Programming Language, "Results from the 2025 go developer survey," https://go.dev/blog/survey2025, 2026, survey conducted Sept. 9-30, 2025; results published 2026; accessed 2026-05-01.

[15] OpenAPI Generator contributors, "Openapi generator usage," https://openapi-generator.tech/docs/usage, 2026, official documentation; accessed 2026-05-01.

[16] Buf, "Buf documentation," https://buf.build/docs/, 2026, official documentation; accessed 2026-05-01.

[17] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "Swe-agent: Agent-computer interfaces enable automated software engineering," *arXiv preprint arXiv:2405.15793*, 2024.

[18] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "Swe-bench: Can language models resolve real-world github issues?" in *International Conference on Learning Representations*, 2024.

[19] OpenTelemetry Authors, "Opentelemetry documentation," https://opentelemetry.io/docs/, 2025, openTelemetry docs landing page, last modified 2025-08-29; accessed 2026-05-01.

[20] ——, "Opentelemetry semantic conventions," https://opentelemetry.io/docs/specs/otel/semantic-conventions/, 2026, openTelemetry documentation, accessed 2026-05-01.

[21] PostgreSQL Global Development Group, "Constraints," https://www.postgresql.org/docs/current/ddl-constraints.html, 2026, postgreSQL documentation; accessed 2026-05-01.

[22] ——, "Row security policies," https://www.postgresql.org/docs/current/ddl-rowsecurity.html, 2026, postgreSQL documentation; accessed 2026-05-01.

[23] Vite Team, "Vite 8.0 is out!" https://vite.dev/blog/announcing-vite8, 2026, vite blog, published 2026-03-12; accessed 2026-05-01.

[24] OpenAI, "Custom instructions with agents.md," https://developers.openai.com/codex/guides/agents-md, 2026, codex documentation, accessed 2026-05-01.

[25] AGENTS.md contributors, "AGENTS.md," https://agents.md/, 2026, agent instruction file specification and guidance; accessed 2026-05-01.

[26] Anthropic, "How claude remembers your project," https://code.claude.com/docs/en/memory, 2026, claude Code documentation, accessed 2026-05-01.

[27] Cursor, "Cursor rules documentation," https://docs.cursor.com/en/context, 2026, cursor documentation, accessed 2026-05-01.

[28] GitHub, "Adding repository custom instructions for github copilot," https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions, 2026, gitHub documentation, accessed 2026-05-01.

[29] SWE-Effi Authors, "Swe-effi: Re-evaluating software ai agent system effectiveness under resource constraints," *arXiv preprint arXiv:2509.09853*, 2025.

[30] OpenHands Authors, "Openhands: An open platform for ai software developers as generalist agents," *arXiv preprint arXiv:2407.16741*, 2024.

[31] Aider Contributors, "Aider: Ai pair programming in your terminal," https://github.com/Aider-AI/aider, 2026, gitHub repository, accessed 2026-05-01.

[32] Microsoft Playwright, "Playwright best practices," https://playwright.dev/docs/best-practices, 2026, playwright documentation, accessed 2026-05-01.

[33] ——, "Writing tests," https://playwright.dev/docs/writing-tests, 2026, official documentation; accessed 2026-05-01.

[34] Storybook, "Visual tests," https://storybook.js.org/docs/9/writing-tests/visual-testing, 2026, official documentation; accessed 2026-05-01.

[35] Microsoft Playwright, "Visual comparisons," https://playwright.dev/docs/test-snapshots, 2026, official documentation; accessed 2026-05-01.

[36] Argos CI, "Argos visual testing," https://argos-ci.com/visual-testing, 2026, product documentation; accessed 2026-05-01.

[37] BackstopJS contributors, "Backstopjs," https://github.com/garris/BackstopJS, 2026, open-source repository; accessed 2026-05-01.

[38] Microsoft Playwright, "Accessibility testing," https://playwright.dev/docs/accessibility-testing, 2026, official documentation; accessed 2026-05-01.

[39] Storybook, "Mocking network requests," https://storybook.js.org/docs/writing-stories/mocking-data-and-modules/mocking-network-requests, 2026, official documentation; accessed 2026-05-01.

[40] World Wide Web Consortium, "Understanding success criterion 2.5.8: Target size (minimum)," https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html, 2026, w3C Web Accessibility Initiative documentation; accessed 2026-05-01.

[41] web.dev, "Optimize cumulative layout shift," https://web.dev/articles/optimize-cls, 2025, last updated 2025-02-07; accessed 2026-05-01.

[42] GitHub, "Working with secret scanning and push protection," https://docs.github.com/en/code-security/secret-scanning/working-with-secret-scanning-and-push-protection, 2026, gitHub documentation; accessed 2026-05-01.

[43] Google OSV, "Osv-scanner," https://google.github.io/osv-scanner/, 2026, official documentation; accessed 2026-05-01.

[44] SLSA Framework contributors, "Supply-chain levels for software artifacts," https://slsa.dev/, 2026, official specification site; accessed 2026-05-01.

[45] M. Nottingham, E. Wilde, and S. Dalal, "Problem details for http apis," https://www.rfc-editor.org/rfc/rfc9457, 2023, rFC 9457.

[46] OpenTelemetry Authors, "Exception attributes," https://opentelemetry.io/docs/specs/semconv/registry/attributes/exception/, 2026, official semantic conventions; accessed 2026-05-01.

[47] MDN Web Docs contributors, "Error: cause," https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/cause, 2026, mDN Web Docs; accessed 2026-05-01.

[48] Rust Project, "Error handling," https://doc.rust-lang.org/stable/rust-by-example/error.html, 2026, rust By Example; accessed 2026-05-01.

[49] anyhow contributors, "anyhow," https://docs.rs/anyhow/latest/anyhow/, 2026, rust crate documentation; accessed 2026-05-01.